PROJECT INTRODUCTION

NYC YELLOW TAXI DATA ANALYTICS.

Large-scale distributed analytics and machine learning across 39.6 million recorded taxi trips.

PROJECT TEAM

Varun Gupta 2023BCS-071
Harsh Patel 2023BCS-024
Rahul Shah 2023BCS-060
Om Sahu 2023BCS-042

TECHNOLOGY

Apache Spark 3.5
Delta Lake 3.0
Scala · Docker · Next.js

THE MISSION

Better decisions for drivers.

We are building actionable intelligence for taxi operators, dispatchers, and urban planners—not simply processing large files.

01 / Understand demand

Show how pickup volume changes across New York City boroughs and across the week.

02 / Locate hotspots

Identify the three busiest pickup zones for every hour of the day.

03 / Study potential tips

Train a model to classify whether a recorded credit-card tip exceeds 20% of the metered fare.

> Goal: make historical demand and recorded trip patterns easier to explore when planning taxi operations.

DATA SOURCE

What are we analyzing?

The New York City Taxi and Limousine Commission (TLC) publishes trip-record datasets from licensed taxi operations.

39.6M
raw trip records

2023–25
source years used in project

265
taxi zone identifiers

One row = one taxi trip

Pickup and drop-off time, pickup and drop-off zone, travel distance, passenger count, fare, payment type, and recorded tip.

How we read the files

Apache Parquet is a compressed, column-oriented file format. Apache Spark divides these records into partitions for parallel computation.

Our source is the New York City Taxi and Limousine Commission (TLC) trip-record dataset. Cash tips are not present in the recorded tip field.

THE PROBLEM

Scale exposes four weaknesses.

A single-machine script cannot reliably solve problems caused by inconsistent, noisy data and costly data movement.

01	**Schema drift**	Historical files disagree on names and types, preventing straightforward reads.
02	**Memory pressure**	Small identifiers stored as wide integers waste working memory.
03	**Faulty telemetry**	Invalid distances, passenger counts, and timestamps bias downstream results.
04	**Network shuffles**	Large joins move records between partitions, increasing runtime and disk spill.

ARCHITECTURE

From raw trips to decisions.

Our five-stage workflow uses Apache Spark for parallel processing and Delta Lake for versioned, reliable intermediate datasets.

01
Ingest
Unify schema & types

→

02
Clean
Filter faults & add features

→

03
Analyze
Join zones & rank demand

→

04
Model
Train tip classifier

→

05
Explore
Next.js dashboard

INPUT

2023–2025 Parquet trip files

INTERMEDIATE STORAGE

Delta Lake tables: `schema_fixed` → `cleaned` → `analytics`

Parquet is a column-oriented data format. Delta Lake adds a transaction log that tracks committed table versions.

STAGE 01 — INGESTION

Smaller types. Less memory.

Schema harmonization converts inconsistent source fields to one agreed naming and type convention.

Before → After

Passenger count	**64-bit → 8-bit**
Payment type	**64-bit → 8-bit**
Pickup / drop-off zone	**64-bit → 16-bit**

ByteType stores one-byte integers; ShortType stores two-byte integers. Conversion must preserve valid source values.

REPORTED MEMORY IMPACT

~40%

reduction in executor memory use after primitive downcasting in the project tests.

Implementation: `TLCSchemaFixerJob.scala` and `ColumnMappingTransf.scala`.

STAGE 02 — CLEANING

Trust the data before the model.

The cleaner removes implausible rides and derives useful time-based attributes before expensive aggregation and training.

Reported cleaning rules

Distance	**0.1–100 miles**
Passengers	**1–6 riders**
Trip duration	**Positive; exclude clock errors**
Missing fee fields	**Fill with zero where appropriate**

RECORDS RETAINED

38.9M

valid trips from 39.6 million raw records.

98.2% data-quality pass rate; approximately 700,000 records removed.

We apply these cleaning rules in `TLCDataCleanerJob.scala` before demand aggregation and model training.

STORAGE DESIGN

Reliable writes by design.

Plain Parquet stores files. Delta Lake adds a transaction log so readers can see a consistent, committed table version.

What could go wrong?

A processing job may fail during an overwrite, leaving partially written files. A downstream task could otherwise observe incomplete results.

What changes with Delta Lake?

A successful transaction updates the `_delta_log` and makes a new version visible. Failed, uncommitted output is not part of the readable snapshot.

Write new data files

Commit transaction log

Expose new table version

ACID means Atomicity, Consistency, Isolation, and Durability. The platform uses Delta Lake 3.0.

STAGE 03 — JOIN OPTIMIZATION

Move the small table. Not the large one.

We join 39.6 million trips to a 265-row zone lookup. A broadcast hash join sends the small lookup to workers instead of redistributing the trips.

STANDARD JOIN

42.8s · 2.84 GB shuffle

```
tripDataDf.join(
  taxiZoneDf,
  col("pickup_location_id") === col("LocationID")
)
```

BROADCAST HASH JOIN

8.4s · 0 MB shuffle

```
tripDataDf.join(
  broadcast(taxiZoneDf),
  col("pickup_location_id") === col("LocationID")
)
```

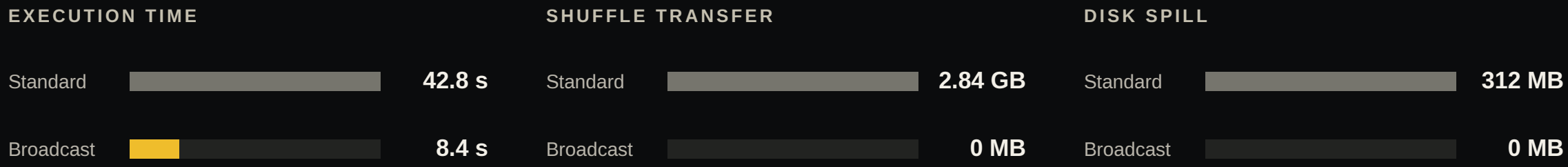
What changed? `broadcast(...)` tells Spark to copy the ~12 KB zone lookup to each worker; the large trip dataset stays in its existing partitions.

Scala excerpt from `TLCAdvancedAnalyticsJob.scala`. We use `pickup_location_id`, the harmonized column name generated in Stage 1.

BENCHMARK

A measurable 5.1× speedup.

Reported benchmark: standard Sort-Merge Join versus Broadcast Hash Join on the project’s Spark execution setup.



Our Spark join benchmark: 42.8s → 8.4s, 2.84 GB → 0 MB shuffle transfer, and 312 MB → 0 MB disk spill. GB = gigabyte; MB = megabyte.

DEMAND INTELLIGENCE

See the weekly demand trend.

Our seven-day rolling average smooths daily peaks and dips separately for each pickup borough.

SPARK SQL WINDOW · SCALA

```
val window7Days = Window.partitionBy("pickup_borough")
  .orderBy("date")
  .rowsBetween(-6, 0)

dailyDemand.withColumn(
  "rolling_7_day_avg", avg("daily_trips").over(window7Days)
)
```

Partition by borough

Each borough gets an independently ordered daily-demand series.

Slide through seven rows

`rowsBetween(-6, 0)` includes the current row and six preceding daily rows.

PICKUP HOTSPOTS

The busiest zones change by hour.

We count pickups per zone and hour, then rank each hour independently to find its leading neighborhoods.

SPARK SQL RANKING · SCALA

```
val windowRank = Window.partitionBy("hour")
  .orderBy(col("total_trips").desc)

hourlyZoneDemand.withColumn("rank", rank().over(windowRank))
  .filter(col("rank") <= 3)
```

08:00

Penn Station

12:00

Midtown Center

22:00

Times Square

These are leading pickup zones in our hourly hotspot analysis. Tied counts may give multiple zones the same `rank()`.

STAGE 04 — MACHINE LEARNING

Train at scale.
Evaluate on unseen trips.

We use Apache Spark's Machine Learning Library (MLlib) to classify whether a recorded tip exceeds 20% of the fare.

FEATURE ASSEMBLY & RANDOM FOREST · SCALA

```
val assembler = new VectorAssembler()
  .setInputCols(Array("trip_distance",
    "trip_duration_in_minutes",
    "passenger_count_d", "hour_d"))
  .setOutputCol("features")

val rf = new RandomForestClassifier()
  .setLabelCol("is_high_tipper")
  .setFeaturesCol("features")
  .setNumTrees(10)
  .setMaxDepth(5)
```

HOLDOUT EVALUATION

0.892

ROC AUC

Area Under the Receiver Operating Characteristic Curve measures how well the classifier separates high-tip from other recorded trips.

80% training · 20% test

Workers calculate split statistics locally; Spark combines summaries to train the forest. ROC AUC of 0.5 corresponds to random ranking.

CONCLUSION

More reliable data.
More useful decisions.

Our engineering approach supports practical insight into taxi demand while making large-scale processing faster and more repeatable.

39.6M → 38.9M
raw to cleaned trips

~40%
reported memory reduction

5.1×
reported join speedup

0.892
reported model ROC AUC

What we built

A repeatable Spark and Delta Lake pipeline with trip cleaning, demand trends, hourly hotspots, and machine-learning exploration.

What comes next

Validate demand estimates on newer records, audit model behavior, and evaluate deployment on a true multi-node cluster.

Our pipeline was implemented in Scala with Apache Spark and Delta Lake, run locally using Docker and orchestrated with PowerShell.